LESSON PLAN · PYTHON (GR 6–8) INTERMEDIATE: PYTHON GAME CODERS · LEVEL 2

Lesson 3: Random Functions and Dice Games

Module 1 | 60 minutes | random, simulation, turn functions | Grades 7–11 · ages 12–16

Objective: Combine random numbers with functions to simulate dice and build a two-player Ludo dice duel with a function for each turn.

Success criteria

- I can write a function that returns a random result.
- I can use a loop to simulate many rolls and count results.
- I can write a rule function and test it with fixed numbers.
- I can split a game turn into its own function.

Key vocabulary

- **random.randint(a, b)**: Gives a random whole number from a to b, including both.
- **simulation**: Using a program to try something many times and see what happens.
- **probability**: How likely something is, like 1 in 6 for rolling a six.
- **game loop**: A loop that keeps the game going until someone wins.
- **turn function**: A function that does one player's whole turn.

You will need

- Projector with Kitabash (lesson video, Python Lab)
- Devices: 1 per learner, with a keyboard
- Printed worksheet (optional)
- Project files: dice_lab.py

Timed lesson plan (60 min)

Time	Stage	What happens
5 min	**Warm-up game: Real Dice Race**	Play Real Dice Race in pairs on a paper number line. List the steps of a turn on the board.
5 min	**Hook and video**	Show the hook. Chat poll: "Guess how many sixes in 600 rolls." Play the video with captions; pause at "Simulate 600 rolls" and compare guesses.
10 min	**Live demo and build-along**	Run the live demo, then everyone types the dice lab. Thumbs check: "Who got between 5 and 15 sixes in 60 rolls?"
8 min	**Challenge, predict and sort**	100-roll experiment, the impossible-output question and the "same or random?" sort. Quick competition: closest guess to the class total of sixes wins.
15 min	**Ludo duel in breakout pairs**	Breakout pairs build the Ludo dice duel: one writes move and tests it, the other writes take_turn; then combine and play.

6 min	**Debug race**	Race to fix “the dice that gave nothing”. The winner explains both bugs.
7 min	**Tournament show-and-tell**	Two volunteers run their duel live with classmates’ names; the class cheers each six. Discuss: does going first help?
4 min	**Wrap-up and exit ticket**	Fist-to-five on the success criteria. Exit ticket: coin() function.

Unplugged warm-up

Real Dice Race (5 min game): in pairs (or the teacher with two volunteers), roll a real or online dice and move counters on a 0–30 number line drawn on paper. A 6 gives a bonus roll and you must land exactly on 30. After two minutes, ask: “What steps happen in every turn?” Write them on the board: roll, move, check finish. That list becomes take_turn.

Teaching the idea

Learners already know random.randint and random.choice from Level 1. Today we put randomness inside functions that return a value, so games read clearly: roll = roll_dice(). Loops let us simulate hundreds of rolls and see patterns (about one six in every six rolls). Splitting a game into small functions, a dice function, a rule function and a turn function, makes it easy to test each part, even though the dice is random.

Analogy: roll_dice() is like a dice cup you can pass round the table: anyone can shake it and it always hands back one number. The rules (move) are the referee who checks every move the same way, no matter who rolled.

Live demo: 1) Type def roll_dice(sides): return random.randint(1, sides) and print roll_dice(6) three times. 2) Ask the class to guess how many sixes in 60 rolls; run the experiment loop and compare. Change to 600 and 6000 and show it gets closer to one sixth. 3) Type move(position, roll) and test move(10, 4) and move(28, 5) live: these give the same answer every time, even though dice are random. 4) Write take_turn(name, position) and call it twice for “Aarav” and “Meera”. 5) Remove return from roll_dice and show the TypeError, then fix it.

Questions to ask

Ask	Listen for
Why put random.randint inside a function?	So we can call roll_dice() whenever we need a roll and the code is easy to read.
In 600 rolls, about how many sixes do you expect?	About 100, one in six.
What should move(28, 5) return if the finish is 30?	28, because 33 would go past the finish.
How can you test a function that uses random?	Test the rule functions with fixed numbers, or use a 1-sided dice so the answer is always 1.

What does take_turn need to return, and why?	The new position, so the main loop can remember where the player is.

Differentiation

Support
Provide move() already written so learners only write take_turn and the loop. Let them start with a finish of 10 so test games are short.

Stretch
Add a snake square and a ladder square with a dictionary later, or three players. Count how many turns the game took and print it.

Common misconceptions

Misconception	How to fix it
A six is due if it has not come up for a while	Each roll is independent: the dice has no memory.
Functions with random cannot be tested	Test the rule functions with fixed numbers and check ranges.
Changing position inside take_turn changes it outside too	Numbers do not change outside the function. Return the new value and store it.
randint(1, 6) never gives 6	randint includes both ends, unlike range.

Assessment

Use the Python Lab results (auto-checked), the quick checks and the Probe answers. Written answers arrive in your review queue.

Exit ticket
Write a function coin() that returns "heads" or "tails" at random.

Homework
Play your Ludo duel ten times with someone at home and tally who wins. Then add a rule of your own, like “a 1 sends you back 2 squares”.

Answer key — Kitabash activities

1. Python Lab: 100-roll experiment (perform · python lab)

Goals checked: Uses the sides parameter · Rolls 100 times · Prints the number of sixes · Uses random, a function and a loop

One solution:

```
import random

def roll_dice(sides):
    return random.randint(1, sides)

sixes = 0
for i in range(100):
    if roll_dice(6) == 6:
        sixes += 1
print("Sixes:", sixes, "out of 100")
```

2. Predict the output: two dice (perform · code output)

3. Will this give the same answer every time you run it? (perform · categorize)

Same every run: 3 + 4, len("ludo"), max(2, 9)
Can change every run: random.randint(1, 6), random.choice(["heads", "tails"]), roll_dice(6) when it uses randint

4. Project: Ludo dice duel (perform · python lab)

Goals checked: move follows the rules · A turn function is used · Someone wins · Uses random, functions and a game loop

Example program:

```
# Ludo dice duel
# Made by: Zoya
import random

FINISH = 30

def roll_dice():
    return random.randint(1, 6)

def move(position, roll):
    if position + roll > FINISH:
        return position
    return position + roll

def take_turn(name, position):
    roll = roll_dice()
    position = move(position, roll)
    print(f"{name} rolled {roll} and is on square {position}")
    if roll == 6 and position < FINISH:
        print("Six! Bonus roll!")
        roll = roll_dice()
        position = move(position, roll)
        print(f"{name} rolled {roll} and is on square {position}")
    return position

print("=== LUDO DICE DUEL: first to square 30 wins! ===")
p1 = input("Player 1 name: ")
p2 = input("Player 2 name: ")
pos1 = 0
pos2 = 0
winner = ""
while winner == "":
    pos1 = take_turn(p1, pos1)
    if pos1 == FINISH:
        winner = p1
    else:
        pos2 = take_turn(p2, pos2)
        if pos2 == FINISH:
            winner = p2
print(f"{winner} wins the Ludo duel!")
```

5. Debug it: the dice that gave nothing (probe · python lab)

Goals checked: Runs without crashing · Prints a real number

One solution:

```
import random

def roll_dice():
    return random.randint(1, 6)

roll = roll_dice()
print("You rolled", roll)
print("Double it:", roll * 2)
```

Answer key — worksheet

1. **Key words:** random.randint(a, b) = Gives a random whole number from a to b, including both. · simulation = Using a program to try something many times and see what happens. · probability = How likely something is, like 1 in 6 for rolling a six. · game loop = A loop that keeps the game going until someone wins.
2. **Plan your program:** Goals: Uses the sides parameter; Rolls 100 times; Prints the number of sixes; Uses random, a function and a loop. One solution: import random def roll_dice(sides): return random.randint(1, sides) sixes = 0 for i in range(100): if roll_dice(6) == 6: sixes += 1 print("Sixes:", sixes, "out of 100")
3. **Predict the output: two dice:** A. 1
4. **Will this give the same answer every time you run it?:** Same every run: 3 + 4, len("ludo"), max(2, 9) · Can change every run: random.randint(1, 6), random.choice(["heads", "tails"]), roll_dice(6) when it uses randint
5. **Plan your program:** Goals: Runs without crashing; Prints a real number. One solution: import random def roll_dice(): return random.randint(1, 6) roll = roll_dice() print("You rolled", roll) print("Double it:", roll * 2)

Kitabash original curriculum.